SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1412 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 40%

Duration :60 Mins

On Class QUESTIONS:5 Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

Annexure A

EXPERIMENTS

Q1. A calculator compiler evaluates arithmetic expressions during parsing using

YACC. (i) Design a YACC grammar for arithmetic operators + and *.

(ii) Embed semantic actions in the grammar rules to compute expression

values. (iii) Evaluate the input expression:

3 + 4 * 5

(iv) Display the computed result after parsing. (10 Marks)

Q2. A compiler constructs syntax trees for subsequent semantic analysis and code generation using YACC.

(i) Define a YACC grammar for arithmetic expressions.

(ii) Implement semantic actions to construct a parse tree / Abstract Syntax Tree

(AST). (iii) Generate and illustrate the tree for:

a + b * c

(iv) Traverse the generated tree and display the traversal order.(10 Marks)

Q3. A compiler evaluates arithmetic expressions using synthesized attributes only in a YACC-based syntax-directed translation scheme.

(i) Implement a Syntax-Directed Definition (SDD) using YACC semantic values ($$, $1, $2, etc.).

(ii) Evaluate the expression:

2 * 3 + 4

(iii) Show how attribute values are propagated during reductions.

(iv) Demonstrate the bottom-up evaluation process performed by the parser. (10 Marks)

Q4. A compiler developed using YACC must verify type compatibility between variables and user-defined types. The system distinguishes between name equivalence and structural equivalence during semantic analysis.

(i) Design a YACC grammar to process type declarations and variable declarations.

(ii) Implement semantic actions and symbol table routines to compare type

expressions.

(iii) Differentiate and verify name equivalence and structural equivalence for the following declarations:

type A = int;
type B = int;
A x;
B y;

(iv) Test the parser with multiple type declarations and assignment statements.

(v) Display the type compatibility result and indicate whether the types are equivalent under:

(a). Name Equivalence

(b). Structural Equivalence (10 Marks)

Q5. A compiler implemented using YACC processes assignment statements involving multiple data types (int, float). During semantic analysis, the compiler performs automatic type conversion (coercion).
(i) Design a YACC grammar for variable declarations and assignment statements.
(ii) Analyze the following program fragment:
float temperature;
int sensor_value;

temperature = sensor_value;
(iii) Implement semantic actions and symbol table entries to support type promotion rules.
(iv) Demonstrate how the compiler performs implicit type conversion (int → float) during assignment.

(v) Display the resulting type information and the final type of the variable after assignment. (10 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q1. Calculator Compiler Using YACC — 10 Marks

(i) YACC Grammar for + and *

The grammar must give * higher precedence than +.

```
yacc
%{
#include <stdio.h>
%}

%token NUMBER
%left '+'
%left '*'
```

```
%%
expr : expr '+' expr
 | expr '*' expr
 | NUMBER
 ;
%%

int main()
 {
 printf("Enter expression: ");
 yyparse();
 return 0;
}

int yyerror(char *s)
 {
 printf("Invalid expression\n");
 return 0;
}
```

(ii) Semantic Actions to Compute Values

```
yacc
%{
#include <stdio.h>
#include <stdlib.h>
%}

%token NUMBER
%left '+'
%left '*'

%%
expr : expr '+' expr { $$ = $1 + $3; }
 | expr '*' expr { $$ = $1 * $3; }
 | NUMBER { $$ = $1; }
 ;
%%
int main()
 {
 printf("Enter expression: ");
 yyparse();
 return 0;
}

int yyerror(char *s)
 {
 printf("Invalid expression\n");
```

```
 return 0;
}
```

(iii) Evaluation

Input:

3 + 4 * 5

According to operator precedence:

4 * 5 = 20

3 + 20 = 23

(iv) Output

Enter expression: 3 + 4 * 5

Result = 23

Evaluation Flow

3 + 4 * 5

↓

4 * 5

↓

20

↓

3 + 20

↓

23

Result

The YACC parser correctly evaluates the arithmetic expression using **semantic actions and operator precedence**.

Q2. Syntax Tree / AST Construction Using YACC — 10 Marks

(i) YACC Grammar

The grammar contains identifiers, addition and multiplication.

yacc

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
 char value[20];
 struct Node *left;
 struct Node *right;
};

struct Node* createNode(char *value, struct Node *l, struct Node
*r) {
 struct Node *n = malloc(sizeof(struct Node));
 strcpy(n->value, value);
```

```
 n->left = l;
 n->right = r;
 return n;
}

void preorder(struct Node *n);
%}

%union {
 char *str;
 struct Node *node;
}

%token <str> ID
%type <node> expr

%left '+'
%left '*'

%%
expr : expr '+' expr
 { $$ = createNode("+", $1, $3); }
 | expr '*' expr
 { $$ = createNode("*", $1, $3); }
 | ID
 { $$ = createNode($1, NULL, NULL); }
 ;
%%

struct Node *root;

void preorder(struct Node *n)
 {
 if(n != NULL)
 {
 printf("%s ", n->value);
 preorder(n->left);
 preorder(n->right);
 }
}
```

(ii) Semantic Actions

The semantic action creates a tree node for every operator.

For:

expr : expr '+' expr

the action is:

C

```
$$ = createNode("+", $1, $3);
```

For multiplication:

C

$$ = createNode("*", $1, $3);
For an identifier:
C
$$ = createNode($1, NULL, NULL);
Thus, $$ represents the newly created parent node.

(iii) AST for a + b * c
Because * has higher precedence:
a + (b * c)
Tree:
```
 +
 / \
 a *
 / \
 b c
```

(iv) Traversal
Preorder
+ a * b c
Inorder
a + b * c
Postorder
a b c * +

Result
Input : a + b * c
AST : + (a, * (b,c))
Preorder : + a * b c
Inorder : a + b * c
Postorder: a b c * +
The AST correctly represents **operator precedence and expression structure**.

Q3. Synthesized Attributes in YACC — 10 Marks

(i) Syntax-Directed Definition
For arithmetic expressions, the value of the left-hand side is calculated from the values of the right-hand side.
yacc
```
%{
#include <stdio.h>
%}

%token NUMBER
%left '+'
%left '*'
```

```
%%
E : E '+' T { $$ = $1 + $3; }
 | T { $$ = $1; }
 ;

T : T '*' F { $$ = $1 * $3; }
 | F { $$ = $1; }
 ;

F : NUMBER { $$ = $1; }
 ;
%%
```

Here:
$$ = value of left-hand side
$1 = value of first symbol
$2 = value of second symbol
$3 = value of third symbol
All attributes are **synthesized attributes** because information moves from children to parent.

(ii) Evaluate 2 * 3 + 4
Input:
2 * 3 + 4
According to precedence:
2 * 3 = 6
6 + 4 = 10
Therefore:
Result = 10

(iii) Attribute Propagation
The reductions occur approximately as follows:
F → 2
F.val = 2

T → F
T.val = 2

F → 3
F.val = 3
T → T * F
T.val = 2 * 3
T.val = 6

E → T
E.val = 6

F → 4
F.val = 4

T → F
T.val = 4

E → E + T
E.val = 6 + 4
E.val = 10

Attribute Table
Reduction Attribute

Value F → 2 2

T → F 2

F → 3 3

T → T * F 6

E → T 6

F → 4 4

T → F 4

E → E + T10

(iv) Bottom-Up
Evaluation 2 * 3 + 4
↓
F → 2
↓
T → 2
↓
F → 3
↓
T → 2 * 3 = 6
↓
E → 6
↓
F → 4
↓
T → 4
↓
E → 6 + 4

↓
E = 10

Result
Input : 2 * 3 + 4
Output : 10
The parser performs **bottom-up evaluation**, and every value is synthesized from the child nodes.

Q4. Name Equivalence and Structural Equivalence — 10 Marks

(i) YACC Grammar
The grammar processes type declarations and variable declarations.
yacc

```
%{
#include <stdio.h>
#include <string.h>

struct Type {
 char name[20];
 char base[20];
};

struct Type types[20];
int count = 0;

void addType(char *name, char *base)
 {
 strcpy(types[count].name, name);
 strcpy(types[count].base, base);
 count++;
}

char* getBase(char *name)
 {
 for(int i = 0; i < count; i++)
 if(strcmp(types[i].name, name) == 0)
 return types[i].base;

 return name;
}
%}
%token TYPE INT ID
%token ';' '='

%%
program : program declaration
 | declaration
 ;
```

```
declaration : TYPE ID '=' INT ';'
 { addType($2, "int"); }
 ;

declaration : ID ';'
 ;
%%
```

(ii) Symbol Table
The symbol table stores user-defined types.
For the given declarations:
type A = int;
type B = int;
Symbol table:

Type Name	Base Type
A	int
B	int

A lookup routine retrieves the underlying type.
C

```
char* getBase(char *name)
 {
 for(int i = 0; i < count; i++)
 if(strcmp(types[i].name, name) == 0)
return types[i].base;

 return name;
}
```

(iii) Name vs Structural
Equivalence Given:
type A = int;
type B = int;

A x;
B y;
Name Equivalence
Under **name equivalence**, two types are equivalent only when they have the same declared type name.
$A \neq B$
Therefore:
A x;

B y;
are **not name equivalent**.
Name Equivalence: NOT EQUIVALENT
Structural Equivalence
Under **structural equivalence**, types are equivalent when their structures are identical.
A → int
B → int
Both have the same underlying structure.
Therefore:
A ≡ B
under structural equivalence.
Structural Equivalence: EQUIVALENT

(iv) Assignment Testing
Example:
A x;
B y;

x = y;
Under Name Equivalence
A ≠ B
Therefore:
Error: Type mismatch
Under Structural Equivalence
A → int
B → int
Therefore:
x = y
is valid.

(v) Output
Type A = int
Type B = int

Name Equivalence:
A and B are NOT EQUIVALENT

Structural Equivalence:
A and B are EQUIVALENT

Assignment:
x = y

Name Equivalence : Type Error
Structural Equivalence : Valid Assignment
Result
The compiler can distinguish between **type identity** and **type structure**. Name equivalence is stricter, whereas structural equivalence compares the actual type structure.

Q5. Type Coercion: Integer to Float — 10 Marks

(i) YACC Grammar

yacc

```
%{
#include <stdio.h>
#include <string.h>

struct Symbol {
 char name[20];
 char type[10];
};

struct Symbol table[20];
int count = 0;

void addSymbol(char *name, char *type)
 {
 strcpy(table[count].name, name);
 strcpy(table[count].type, type);
 count++;
}

char* getType(char *name)
 {
 for(int i = 0; i < count; i++)
 if(strcmp(table[i].name, name) == 0)
 return table[i].type;

 return "unknown";
}
%}

%token FLOAT INT ID
%token '=' ';'

%%
program : program statement
 | statement
 ;

statement : FLOAT ID ';'
 { addSymbol($2, "float"); }
 ;
statement : INT ID ';'
 { addSymbol($2, "int"); }
 ;
```

```
statement : ID '=' ID ';'
 {
 printf("Assignment: %s = %s\n", $1, $3);
 }
 ;
%%
```

(ii) Program Fragment

```
float temperature;
int sensor_value;

temperature = sensor_value;
```

Symbol table:

Variable	Type
temperature	float
sensor_value	int

(iii) Type Promotion Rules

The compiler checks the type of the left-hand side and right-hand side. LHS = float
RHS = int
The promotion rule is:
int → float
Therefore, the integer value can be automatically converted to a floating-point value. Example:
sensor_value = 25
becomes:
(float)sensor_value = 25.0

(iv) Implicit Type Conversion

Assignment:
temperature = sensor_value;
Type checking:
temperature → float
sensor_value → int
The compiler performs:
int sensor_value
↓
type conversion
↓
float sensor_value
↓
temperature

Equivalent internal operation:
temperature = (float)sensor_value;
This is called **implicit type coercion**.

(v) Output
Symbol Table

temperature float
sensor_value int

Assignment:
temperature = sensor_value

LHS Type : float
RHS Type : int

Type Conversion:
int → float

Coercion Applied:
sensor_value converted from int to float

Final Assignment Type : float
Assignment Status : Valid

Result
The compiler successfully performs **implicit widening conversion from** int **to** float, so the assignment is type compatible.